Mattermost Cross Guard Threat Model

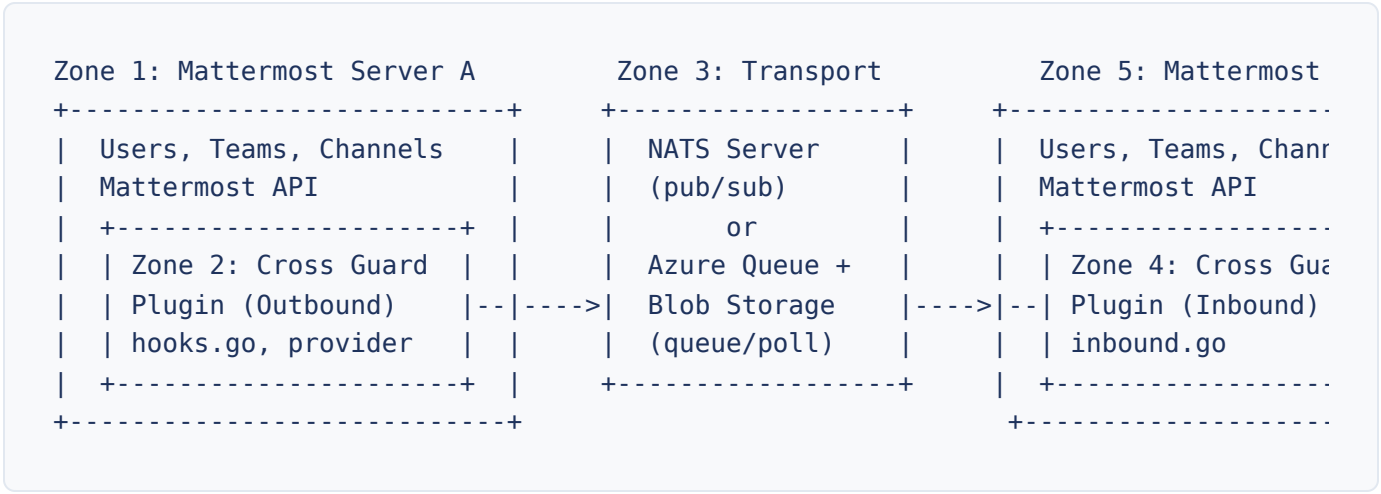
Security analysis of Cross Guard's cross-domain relay architecture

Introduction

Cross Guard operates as a Cross Domain Solution (CDS), bridging two or more Mattermost instances across security domain boundaries using NATS or Azure Queue Storage as the transport. This page documents the trust boundaries, the data that crosses them, and the threats and mitigations at each layer.

Intended audience: security reviewers, accreditors, and administrators evaluating Cross Guard for deployment in classified or sensitive environments.

System Architecture and Trust Zones



Zone	Description	Trust Level	Controlled By
Zone 1: Mattermost Server A	Source Mattermost instance. Users, teams, channels, posts. Authenticated via Mattermost session management.	Trusted	Server A admin

Zone	Description	Trust Level	Controlled By
Zone 2: Cross Guard Plugin (Outbound)	Plugin code running inside Server A. Hooks into post lifecycle events (`MessageHasBeenPosted` , `MessageHasBeenUpdated` , `MessageHasBeenDeleted` , reaction hooks in `hooks.go`). Serializes to message envelope, publishes via `connections.go:publishToOutbound()` .	Trusted (same process as Mattermost)	Plugin code + Server A admin config
Zone 3: Transport Layer (NATS)	External message broker. Receives published messages, delivers to subscribers. Can be shared or dedicated. May be in a separate network segment.	Semi-trusted (transport security only)	NATS admin (may be separate from MM admin)
Zone 3 (alt): Azure Queue Storage	Azure-managed queue and blob storage. Messages are enqueued by the outbound plugin and polled by the inbound plugin. File attachments are stored in Azure Blob Storage containers.	Semi-trusted	Azure subscription owner. Data encrypted at rest and in transit by Azure. Access controlled via connection string (shared key).
Zone 4: Cross Guard Plugin (Inbound)	Plugin code running inside Server B. Subscribes to NATS, deserializes envelopes (`inbound.go:handleInboundMessage()`), resolves teams/channels/users, creates local posts.	Trusted (same process as Mattermost)	Plugin code + Server B admin config
Zone 5: Mattermost Server B	Destination Mattermost instance. Receives relayed messages as locally-created posts.	Trusted	Server B admin

Data Crossing the Domain Boundary

The following data structures from `server/model/` transit the transport layer (NATS bus, Azure Queue Storage, Azure Blob Storage, or Azure Service Bus) between trust zones. Understanding exactly what crosses the boundary is critical for CDS evaluation.

Envelope (`model/message.go:Envelope`)

The envelope is a single-layer structure with the payload embedded directly (not as a serialized string). Messages may be serialized as JSON or XML depending on the outbound connection's `message_format` setting. Inbound format is auto-detected by `model.DetectFormat()`. The XML wire format is defined by `schema/crossguard.xsd` (namespace `urn:mattermost-crossguard`); 12 schema-valid round-trip-tested example payloads (one per message-type variant) live under `schema/examples/` and are byte-for-byte identical to what the plugin puts on the wire, with that contract enforced by `TestExampleFiles_Symmetric` in `server/model/examples_roundtrip_test.go`.

`type`	Message type constant (e.g., `crossguard_post`)
`timestamp`	RFC 3339 UTC timestamp
`post_message`	Post or update payload (present when type is `crossguard_post` or `crossguard_update`)
`delete_message`	Delete payload (present when type is `crossguard_delete`)
`reaction_message`	Reaction payload (present when type is `crossguard_reaction_add` or `crossguard_reaction_remove`)
`test_message`	Test payload (present when type is `crossguard_test`)

PostMessage Payload (`crossguard_post`, `crossguard_update`)

Field	Source	User-Controlled?	Used On Receive For
`post_id`	`post.Id` (26-char Mattermost ID)	No	Post mapping (KV store key)
`root_id`	`post.RootId`	No	Thread parent resolution

Field	Source	User-Controlled?	Used On Receive For
`channel_id`	`post.ChannelId`	No	Not used directly (name-based routing)
`channel_name`	`channel.Name`	Indirectly (admin-set)	**Routing:** `GetChannelByName()`
`team_id`	`channel.TeamId`	No	Not used directly
`team_name`	`team.Name`	Indirectly (admin-set)	**Routing:** `GetTeamByName()` **or rewrite lookup**
`user_id`	`post.UserId`	No	Not used on receive
`username`	`user.Username`	Yes (users choose usernames)	**User resolution:** `resolveInboundUser()`
`message`	`post.Message`	**Yes** (full post content)	Post body text
`create_at`	`post.CreateAt`	No	Not used on receive

DeleteMessage Payload (`crossguard_delete`)

Fields: `post_id`, `channel_id`, `channel_name`, `team_id`, `team_name`

ReactionMessage Payload (`crossguard_reaction_add`, `crossguard_reaction_remove`)

All PostMessage fields except `message`, `root_id`, `create_at`; adds `emoji_name`.

Data Intentionally Excluded from Relay

> **Good security hygiene**
> The following data is intentionally excluded from the relay envelope: post properties (`post.Props`, except the internal `crossguard_relayed` flag), user metadata (email, profile image, custom fields), channel purpose/header, hashtags, pinned status, edit history, and interactive message elements (buttons, menus). File attachments (`post.FileIds`) are not included in the envelope but can be

> transferred separately via NATS JetStream Object Store, Azure Blob Storage (a paired sidecar container for the Azure Queue and Azure Service Bus providers, or the same container as the WAL for the Azure Blob provider), depending on the configured provider, when `file_transfer_enabled` is true on the connection. File transfer is disabled by default and supports allow/deny type filtering and size limits.

Threat Analysis (STRIDE)

Each threat is identified by a unique ID, categorized by STRIDE, and assessed for severity. Code references point to the specific source files where the behavior occurs.

Spoofing

T-SPOOF-1: Username Impersonation via Username Lookup

Severity	MEDIUM
Description	When `UsernameLookup` is enabled (default: `true`), `resolveInboundUser()` in `sync_user.go` looks up a local user by the `username` field from the inbound message. If a match is found, the relayed post is created under that real local user's ID. An attacker controlling the remote Mattermost instance (or with NATS write access) can forge messages that appear to come from any local user whose username they know.
Code Path	`inbound.go` calls `resolveInboundUser()`, which calls `p.API.GetUserByUsername(username)` in `sync_user.go`. If found and not a sync user, returns the real user ID.
Mitigation	1. Disable `UsernameLookup` in plugin settings to force all inbound messages through synthetic sync users (e.g., `alice.relay-from-a`). Sync users are visually distinguishable by their `LastName` field showing `(via connName)`. 2. Restrict NATS server network access to only authorized Mattermost servers using IPsec, firewall rules, or network segmentation. Configure NATS ACLs so only authorized Mattermost server clients can publish to `crossguard.*` subjects.
Residual Risk	When `UsernameLookup` is enabled, impersonation remains possible from a compromised but network-authorized Mattermost server. The combination of network-level restrictions and NATS ACLs significantly reduces the attack surface compared to an open NATS bus.

T-SPOOF-2: NATS Message Injection (No Per-Message Authentication)

Severity	MEDIUM
Description	The NATS envelope (`model/message.go:Envelope`) contains no cryptographic signature, HMAC, or message authentication code. Any entity with publish access to the NATS subject can inject arbitrary messages (in either JSON or XML format) that the inbound handler will process as legitimate.
Code Path	`inbound.go` calls `model.DetectFormat(msg.Data)` followed by `model.Unmarshal()` with no signature verification. The deserialized envelope is dispatched directly to handlers.
Mitigation	1. TLS encryption (`nats_provider.go` , min TLS 1.2) protects against passive eavesdropping. NATS authentication (token or credentials) restricts who can connect. 2. Restrict NATS server network access to only authorized Mattermost servers using IPsec, firewall rules, or network segmentation. Configure NATS ACLs so only authorized Mattermost server clients can publish to `crossguard.*` subjects.
Residual Risk	A compromised but network-authorized Mattermost server with valid NATS credentials can still inject messages. No end-to-end integrity verification exists, but the combination of network-level restrictions and NATS ACLs significantly limits the attack surface.

T-SPOOF-AZ1: Azure Credential Compromise

Severity	HIGH
Description	An attacker who obtains the Azure Storage connection string or shared-key (Azure Queue or Azure Blob providers), the Service Bus SAS connection string (Azure Service Bus provider), or the Service Principal client secret (any of the three providers when `auth_mode` is `service-principal`) can send or read messages on the corresponding transport. Shared-key credentials provide full account access; a SAS connection string built from `RootManageSharedAccessKey` grants full namespace control, while a scoped SAS rule grants only the rights it was created with. A Service Principal credential grants exactly the Azure RBAC roles assigned to the SP at the configured scope. Mattermost does not encrypt plugin settings at rest: by default the connections JSON (containing the shared-key, SAS, or `client_secret` value) lives as plaintext in `config.json` on disk and in the Mattermost configuration database, where any operator with disk, backup, or DB access can read it. The Mattermost `GET /api/v4/config` endpoint also returns these secrets in cleartext to any System Admin.

Mitigation	1. **Prefer Service Principal over shared-key / SAS.** An SP credential can be scoped via Azure RBAC to the minimum data-plane role required (e.g., `Storage Queue Data Contributor` on one storage account, `Azure Service Bus Data Sender` + `Receiver` on one queue) instead of granting account-wide or namespace-wide control, and rotation happens in Azure AD without touching plugin config. See the admin guide's Azure Service Principal Authentication section for the minimum RBAC roles and Phase 1 deployment guidance. 2. **Keep the SP secret out of plugin config on disk.** Inject the entire connections JSON via Mattermost's plugin-settings env-var substitution (`MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` and the inbound counterpart). That env var can be sourced from a Kubernetes Secret, a CSI volume + init container, systemd `LoadCredential=`, or HashiCorp Vault. With this in place the `client_secret` never lands in `config.json` on disk, in the configuration database, or in a `GET /api/v4/config` response. 3. **Inject the connections JSON via environment variables instead of saving it through the System Console.** Mattermost substitutes plugin settings from environment variables of the form `MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` and `MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_INBOUNDCONNECTIONS` at config-load time. The credential then lives only in the Mattermost process environment (and any orchestrator-managed secret store such as Kubernetes secrets, systemd `LoadCredential=`, or HashiCorp Vault that produced it), not in `config.json` on disk and not in the Mattermost configuration database. This is the recommended pattern for CDS deployments and is compatible with both legacy shared-key/SAS and Service Principal credentials. 4. Prefer scoped Service Bus SAS rules over the namespace-wide `RootManageSharedAccessKey` when using connection-string mode, so an exposed credential grants only the minimum needed rights (Send / Listen / Manage scoped to one queue). 5. Rotate Azure Storage account keys, Service Bus SAS keys, and Service Principal client secrets on a regular schedule. SP rotation happens in Azure AD and is picked up at the next plugin construction (plugin reload or Mattermost restart); shared-key/SAS rotation requires updating the env-var-managed secret as well. 6. The Azure providers sanitize credential material out of error messages (account keys, SAS query params, and SP secret values via the shared `sanitizeAzureError()` path) before logging or returning to the admin UI, so a compromised log file does not leak the credential. The save-time SP probe applies the same sanitization to AAD errors. 7. Use Azure Private Endpoints to restrict network access to the storage account and Service Bus namespace. Monitor Azure Storage analytics and Service Bus diagnostic logs for anomalous access patterns. For SP, also monitor Azure AD sign-in logs for the SP's `appId` to detect unexpected token issuance.

Residual Risk	With env-var injection of the connections JSON, the credential is not on disk or in the Mattermost DB, but it is still readable by anyone who can read the Mattermost process environment (root on the host, container exec, `/proc/<pid>/environ`, container orchestrator logs). Without that indirection, the credential sits in plaintext in `config.json`, in the configuration database, and in any `GET /api/v4/config` response made by a System Admin, exposing every operator with disk, backup, DB, or System Admin access. The "secret in plugin config" exposure is known and tracked as Phase 1B; the structural fix replaces the cleartext with a reference. In all cases, a compromised credential grants the access tied to its scope (account-wide for shared-key, namespace-wide for unscoped SAS, role-scoped for SP) until rotated.

Tampering

T-TAMP-1: Unauthorized Post Edit Propagation

Severity	MEDIUM
Description	`handleInboundUpdate()` in `inbound.go` looks up a local post by remote post ID mapping and replaces its `Message` field. There is no verification that the update came from the original author or that the remote user had edit permissions.
Code Path	`inbound.go` retrieves mapping, sets `existing.Message = postMsg.MessageText`, calls `UpdatePost()`.
Mitigation	Post mapping keys (`pm-{connName}-{remotePostId}`) use connection-scoped namespaces, limiting edits to the connection that created the original post. The remote Mattermost server enforces its own edit permissions before relaying.
Residual Risk	If transport credentials are compromised (NATS token/password or Azure connection string), fabricated update messages can modify any previously relayed post.

T-TAMP-2: Unauthorized Post Deletion

Severity	MEDIUM
Description	`handleInboundDelete()` in `inbound.go` deletes a local post based solely on the remote post ID in the inbound message. No author or permission verification.
Code Path	`inbound.go` retrieves mapping, calls `DeletePost(localPostID)`.
Mitigation	Same connection-scoped namespace as edits. The deleting flag mechanism (`SetDeletingFlag` / `ClearDeletingFlag`) prevents delete loops but does not

	authenticate the delete request.
Residual Risk	Same as T-TAMP-1.

T-TAMP-3: Post Mapping Overwrite

Severity	LOW
Description	`SetPostMapping()` in `inbound.go` stores `connName:remotePostId -> localPostId`. The inbound handler checks for existing post ID mappings before creating a post. While duplicate remote post IDs are detected and skipped via an idempotency check, a race condition between the check and the store write could theoretically allow a mapping conflict under extremely high concurrency.
Mitigation	The idempotency check (`GetPostMapping`) prevents replay of previously-seen post IDs. Post IDs are 26-character random strings (Mattermost `NewId()`), making collision unlikely without intentional targeting. Connection-scoped namespacing limits blast radius.

T-TAMP-AZ1: Azure Transport Message Tampering

Severity	MEDIUM
Description	The envelope is not signed or encrypted by the application on any Azure transport: messages in Azure Queue Storage are Base64-encoded, batched WAL blobs in the Azure Blob provider are length-prefixed plaintext envelopes, and Azure Service Bus AMQP message bodies are raw envelope bytes. An attacker with sufficient access to the underlying store (storage account or Service Bus namespace) could modify in-flight or at-rest messages before they are consumed. The Azure Service Bus provider's blob sidecar (file attachments) inherits the same exposure as the Azure Blob provider.
Mitigation	Restrict storage account and Service Bus namespace access via Azure RBAC and network rules. Prefer scoped SAS tokens (Storage) and scoped Service Bus SAS rules over root credentials. Azure Storage and Azure Service Bus both provide transport-level integrity via TLS. Azure Service Bus's PeekLock with `DeliveryCount` limits how many times a tampered message can be redelivered before the broker auto-moves it to the dead-letter sub-queue.

Repudiation

T-REPUD-1: No Audit Trail for Relayed Actions

Severity	MEDIUM
Description	Relayed posts, edits, deletes, and reactions are created via the Mattermost Plugin API. While Mattermost logs post creation, there is no Cross Guard-specific audit log linking a local action back to the specific transport message, remote server, or remote user that triggered it.
Mitigation	Plugin logs contain connection name, remote post IDs, and usernames at DEBUG/WARN level. Team init/teardown actions post bot announcements to `#town-square` with the acting user's name. The `crossguard_relayed` post property marks all inbound posts.
Residual Risk	Forensic reconstruction requires correlating logs across both Mattermost instances and the transport infrastructure (NATS server or Azure Storage account).

T-REPUD-AZ1: Limited Azure Audit Trail

Severity	LOW
Description	Azure Storage and Azure Service Bus both provide access logging, but correlating queue, blob, or AMQP operations to specific Cross Guard messages requires additional effort. Azure Queue message content is Base64-encoded in logs; Azure Blob batch blobs are opaque containers from the audit log's perspective; Azure Service Bus diagnostic logs record send/receive operations and message metadata but not application payloads.
Mitigation	Enable Azure Storage Analytics logging on the storage account, and enable Azure Service Bus diagnostic settings for the namespace. Enable Azure Monitor diagnostic settings on both. Cross Guard logs all inbound/outbound operations with connection names, post IDs, and (for Service Bus) `DeliveryCount` warnings via error code `26004`.

Information Disclosure

T-INFO-1: Message Content Visible to NATS Server

Severity	MEDIUM (in classified environments)
Description	TLS encrypts data in transit between plugin and NATS server, but the NATS server itself processes messages in cleartext. A compromised or malicious NATS server operator can read all relayed message content.

Mitigation	1. Use a dedicated, locally-administered NATS server. Enable TLS with mutual certificate authentication (`client_cert` / `client_key` in `nats_provider.go`). For CDS deployments, the NATS server should be within a controlled enclave. 2. Restrict NATS server network access to only authorized Mattermost servers using IPsec, firewall rules, or network segmentation. Configure NATS ACLs so only authorized Mattermost server clients can subscribe to `crossguard.*` subjects.
Residual Risk	No application-layer encryption. The NATS server still sees plaintext payloads, but network-level restrictions and ACLs limit exposure to only authorized infrastructure.

T-INFO-2: Team/Channel Topology Leakage

Severity	LOW
Description	Message envelopes contain `team_name` and `channel_name` from the source server. An entity with transport read access (NATS subscribe or Azure queue dequeue) learns the team and channel structure of the sending server.
Mitigation	Use dedicated transport channels per connection pair. For NATS, restrict ACLs. For Azure, restrict storage account access via RBAC and network rules.

T-INFO-3: Status Endpoint Data Exposure

Severity	LOW
Description	`GET /api/v1/teams/{team_id}/status` and `GET /api/v1/channels/{channel_id}/status` are accessible to any team/channel member (not just admins). These reveal connection names, directions, and remote team name rewrites.
Mitigation	Global status (`GET /api/v1/status`) is restricted to System Admins. Connection credentials (tokens, passwords, cert paths) are redacted from all status responses.

T-INFO-4: NATS Credentials in Plugin Configuration

Severity	LOW
Description	NATS tokens and passwords are stored as plaintext strings in the plugin settings JSON (`configuration.go`). Any System Admin can view them in the System Console.

Mitigation	Only System Admins can access plugin configuration. Credentials are redacted in status API responses.

T-INFO-AZ1: Connection String Exposure in Plugin Config

Severity	MEDIUM
Description	Azure transports store credentials in the plugin's connections JSON: the Azure Storage connection string (or account name plus shared key) for the Azure Queue and Azure Blob providers, and a Service Bus SAS connection string for the Azure Service Bus provider (with a separate Storage account credential for its blob sidecar). Mattermost does not encrypt plugin settings at rest, so by default these credentials are stored as plaintext in `config.json` on disk and in the Mattermost configuration database. System Admins with System Console access can view them; any operator with disk, backup, or DB access can also read them.
Mitigation	1. Inject the connections JSON via the Mattermost env-var pattern (`MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` and `MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_INBOUNDCONNECTIONS`) sourced from a secret store (Kubernetes secrets, systemd `LoadCredential=`, HashiCorp Vault). This keeps the credential out of `config.json` and the Mattermost configuration database. See T-SPOOF-AZ1 for details. 2. Limit System Admin access; the System Console still allows reading of the in-memory plugin settings. 3. Rotate Azure Storage account keys and Service Bus SAS keys on a regular schedule. Update the env-var-managed secret after rotation. 4. Prefer scoped Service Bus SAS rules over `RootManageSharedAccessKey` so an exposed credential grants only the minimum needed rights.

Denial of Service

T-DOS-1: Inbound Message Flood

Severity	MEDIUM
Description	The inbound handler uses a semaphore of 256 concurrent goroutines (`plugin.go` `relaySem`). An attacker flooding the NATS subject can fill the semaphore, causing legitimate messages to be dropped with "Relay semaphore full, dropping inbound message" warnings.

Code Path	`inbound.go`, semaphore channel check with `default:` drop.
Mitigation	The semaphore prevents unbounded resource consumption. NATS server rate limiting can be configured externally. Connection authentication limits who can publish.
Residual Risk	No per-connection or per-subject rate limiting within the plugin itself.

T-DOS-2: Outbound Retry Amplification

Severity	LOW
Description	`publishToOutbound()` in `connections.go` calls the provider's Publish method. The NATS provider retries failed publishes 3 times with exponential backoff (500ms, 1s, 2s). If the transport is unreachable, each outbound message blocks a goroutine for up to ~3.5 seconds. The Azure provider does not retry at the application level.
Mitigation	Context cancellation (`ctx.Done()`) allows graceful shutdown. Messages are dropped (not queued) after retry exhaustion.

T-DOS-AZ1: Azure Transport Saturation

Severity	MEDIUM
Description	An attacker with write access to an Azure transport (Queue, Blob container, or Service Bus queue), or a misconfigured outbound server, could flood the transport with messages, causing the inbound side to process excessive messages and potentially overwhelming the Mattermost server.
Mitigation	Azure Queue Storage and Azure Service Bus both have built-in throttling. The inbound side bounds processing rate per provider: Azure Queue polls at most 32 messages per batch with a 5-second interval, the Azure Blob provider's batch reads are bounded by `flush_interval_seconds` and per-blob lease, and the Azure Service Bus provider uses PeekLock receive (batch up to 32, 5-second max wait) plus server-side `DeliveryCount` with automatic dead-letter on `MaxDeliveryCount` exceedance. The Service Bus DLQ contains poison messages without a custom retry-tracking layer; the plugin logs a WARN with error code `26004` when `DeliveryCount > 1` so operators can detect buildup before DLQ moves happen. Monitor queue depth, blob container size, and Service Bus active/DLQ counts via Azure metrics. Set up alerts for unusual growth.

Elevation of Privilege

T-PRIV-1: Connection Management by Non-Admins

Severity	MEDIUM
Description	By default (`RestrictToSystemAdmins=false`), Team Admins can link/unlink connections to their teams, and Channel Admins can manage channel connections. A compromised or rogue Team Admin could enable relay on channels without broader organizational approval.
Code Path	`command.go` (`isTeamAdminOrSystemAdmin`) checks `RestrictToSystemAdmins` .
Mitigation	Enable `RestrictToSystemAdmins` setting to lock all connection management to System Admins only. Alternatively, enable both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` to allow Team Admins to submit link requests that require System Admin approval via interactive DM buttons, while still allowing direct unlinking. Inbound connections require explicit Accept via admin prompts before messages flow.
Residual Risk	With the setting disabled, the blast radius is limited to the Team Admin's own team. With request mode enabled, Team Admins can unlink connections directly without approval, but linking always requires System Admin approval.

T-PRIV-2: Sync User Auto-Membership

Severity	MEDIUM
Description	`ensureMembership()` in `sync_user.go` automatically adds sync users (and real users when username lookup is enabled) to teams and channels as messages arrive. This bypasses normal Mattermost invitation/join workflows.
Code Path	`sync_user.go` calls `ensureMembership(user.Id, teamID, channelID)` , which calls `CreateTeamMember` and `AddChannelMember` .
Mitigation	Only affects channels/teams that have been explicitly linked to an inbound connection by an admin. The `system_user` role assigned to sync users provides no admin privileges.

T-ELEV-AZ1: Blob Container Public Access Misconfiguration

Severity	HIGH
Description	If an Azure Blob container is accidentally configured with public access (blob or container level), its contents become accessible to anyone on the internet without

	authentication. This applies to file attachment containers used by the Azure Queue and Azure Service Bus providers and to the combined WAL-plus-files container used by the Azure Blob provider. The Azure Service Bus provider's blob sidecar uses an independent storage account and so must be audited separately from the Service Bus namespace.
Mitigation	The plugin creates containers with private access by default. Enforce "no public access" at the storage account level via Azure policy. Audit container access levels regularly across every storage account in scope (including the independent Service Bus blob sidecar account). Use Azure Defender for Storage to detect public access misconfigurations.

Relay Loop Prevention

Loop prevention is critical for CDS operation. Cross Guard uses multiple independent mechanisms to ensure a relayed message is never re-relayed back to its source.

Primary: `crossguard_relayed` Property

- Inbound posts are created with `post.AddProp("crossguard_relayed", true)` in `inbound.go`
- Outbound hooks check `post.GetProp("crossguard_relayed") != nil` in `hooks.go` and skip if present
- This prevents a relayed post from being re-relayed back to the source

Secondary: Bot User Filtering

- All outbound hooks check `post.UserId == p.botUserID` in `hooks.go` and skip bot posts
- Bot announcements (init/teardown messages) never relay

Tertiary: Sync User Filtering for Reactions

- `hooks.go` checks `user.Position == "crossguard-sync"` to prevent sync user reactions from relaying back

Delete-Specific: Deleting Flag

- `inbound.go` sets `SetDeletingFlag(localPostID)` before calling `DeletePost`

- `hooks.go` checks `IsDeletingFlagSet(post.Id)` in `MessageHasBeenDeleted` hook
- If set, the delete is not re-relayed, preventing infinite delete loops
- Flag is cleared after deletion completes

NATS Transport Security

TLS Configuration

- Minimum TLS 1.2 enforced (`tls.VersionTLS12`)
- Optional mutual TLS: client certificate + key pair
- Optional CA certificate for server verification
- Both `client_cert` and `client_key` must be provided together (validated)

Authentication Methods

Method	Configuration	Description
`none`	No additional fields	No NATS authentication
`token`	`token` field	NATS token-based auth
`credentials`	`username` + `password`	Username/password auth

Connection Resilience

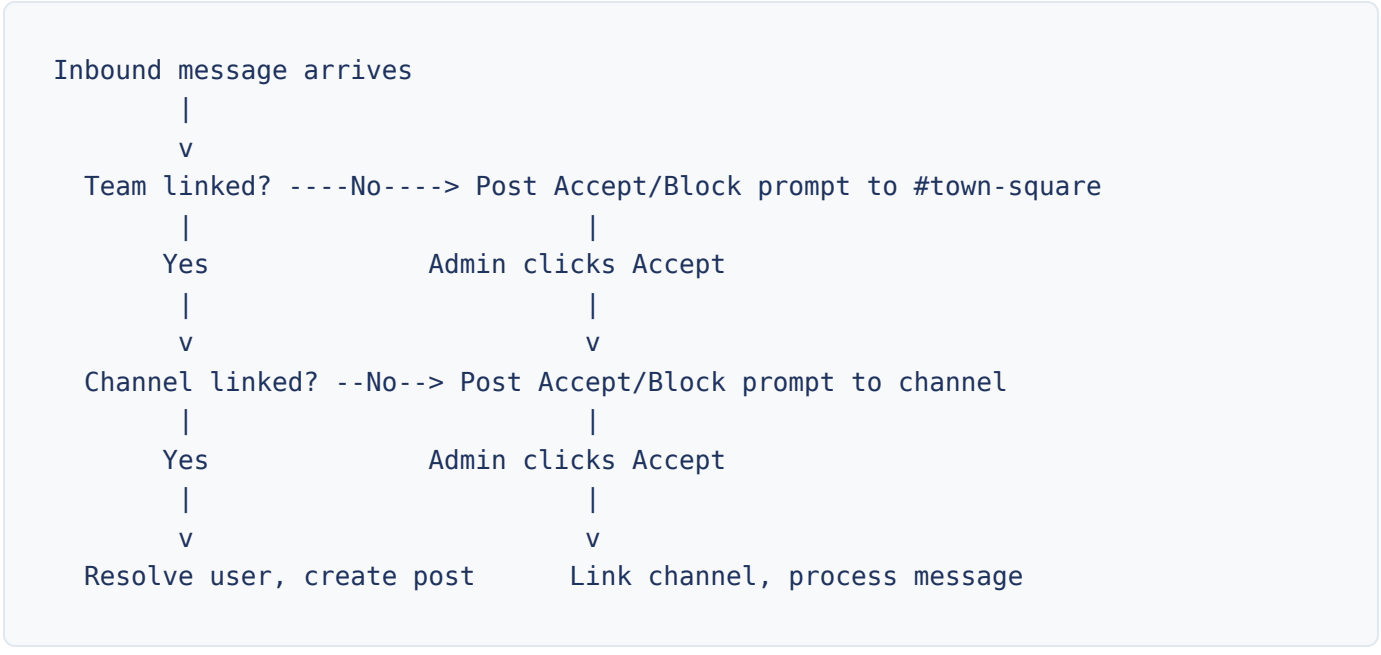
- Persistent connections with unlimited reconnects (`natsMaxReconnects = -1`)
- 2-second reconnect wait
- Disconnect/reconnect event logging

Recommendations by Deployment Sensitivity

Deployment	TLS	Auth	Mutual TLS	Dedicated NATS	NATS ACLs
Development	Optional	Optional	No	No	No
Production (standard)	Required	Token or credentials	Recommended	Recommended	Recommended
CDS (classified)	Required	Credentials	Required	Required	Required

Inbound Connection Acceptance Flow

The prompt-based acceptance workflow prevents unauthorized inbound relay. No messages flow until an admin explicitly accepts the connection at both the team and channel level.



1. Inbound message arrives for an unlinked team
2. `handleUnlinkedInbound()` posts interactive prompt to `#town-square` with Accept/Block buttons
3. Team Admin or System Admin clicks Accept (calls `POST /api/v1/prompt/accept`)
4. Handler verifies `isTeamAdminOrSystemAdmin()` permission
5. Connection is linked to team, prompt deleted

6. Same flow repeats at channel level
7. Blocked prompts can be cleared with `/crossguard reset-prompt`

Deployment Recommendations for CDS Environments

> **Hardened Configuration Checklist**
> The following settings are recommended for Cross Guard deployments in classified or sensitive environments.

1. **Enable `RestrictToSystemAdmins`**: Lock all connection management to System Admins. Optionally enable `AllowTeamAdminRequests` to let Team Admins submit link requests that require System Admin approval.
2. **Disable `UsernameLookup`**: Force all inbound messages through sync users to prevent impersonation.
3. **Enable TLS with mutual authentication**: Use client certificates on all NATS connections.
4. **Dedicated NATS server**: Do not share the NATS server with other applications.
5. **NATS ACLs**: Restrict publish/subscribe permissions per subject per client.
6. **Separate subjects per direction**: Use different NATS subjects for each relay direction (e.g., `crossguard.a-to-b` and `crossguard.b-to-a`) to prevent echo.
7. **Network segmentation**: Place NATS server in a controlled network segment between the two Mattermost instances.
8. **Monitor plugin logs**: Watch for "Relay semaphore full", "Unknown inbound message type", and failed resolve warnings as indicators of anomalous activity.
9. **Review sync users periodically**: Audit users with `Position="crossguard-sync"` to detect unexpected user creation.
10. **Use `crossguard.` subject prefix enforcement**: All subjects must start with `crossguard.` (validated in `configuration.go`), preventing accidental subscription to unrelated NATS traffic.

Threat Summary Matrix

ID	Threat	Category	Severity	Mitigated?	Control
T-SPOOF-1	Username impersonation	Spoofing	MEDIUM	Partial	Disable `UsernameLookup` + NATS network/ACL restrictions
T-SPOOF-2	NATS message injection	Spoofing	MEDIUM	Partial	TLS + NATS auth + network/ACL restrictions
T-TAMP-1	Unauthorized edit propagation	Tampering	MEDIUM	Partial	Connection-scoped mapping
T-TAMP-2	Unauthorized delete propagation	Tampering	MEDIUM	Partial	Connection-scoped mapping
T-TAMP-3	Post mapping overwrite	Tampering	LOW	By design	Random 26-char IDs
T-REPUD-1	No audit trail	Repudiation	MEDIUM	Partial	Plugin logs + bot announcements
T-INFO-1	NATS sees cleartext	Info Disclosure	MEDIUM	Partial	Dedicated NATS + mTLS + network/ACL restrictions
T-INFO-2	Topology leakage	Info Disclosure	LOW	Partial	Dedicated transport channels + ACLs/RBAC
T-INFO-3	Status data to members	Info Disclosure	LOW	By design	Credentials redacted
T-INFO-4	Config credentials	Info Disclosure	LOW	By design	System Admin only

ID	Threat	Category	Severity	Mitigated?	Control
T-DOS-1	Inbound message flood	DoS	MEDIUM	Partial	Semaphore (256) + NATS rate limiting
T-DOS-2	Outbound retry amplification	DoS	LOW	By design	3 retries, context cancellation
T-PRIV-1	Non-admin connection mgmt	Privilege	MEDIUM	Config option	Enable `RestrictToSystemAdmins` (optionally with `AllowTeamAdminRequests`)
T-PRIV-2	Sync user auto-membership	Privilege	MEDIUM	By design	Limited to linked channels
T-SPOOF-AZ1	Azure credential compromise (Storage and Service Bus SAS)	Spoofing	HIGH	Partial	Env-var injection + key rotation + Private Endpoints + scoped SAS rules + SAS error sanitization
T-TAMP-AZ1	Azure transport message tampering (Queue, Blob, Service Bus)	Tampering	MEDIUM	Partial	Azure RBAC + network rules + TLS transport integrity + Service Bus DeliveryCount/DLQ
T-REPUD-AZ1	Limited Azure audit trail	Repudiation	LOW	Partial	Azure Storage Analytics + Service Bus diagnostic logs + plugin logs
T-INFO-AZ1	Connection string exposure in config	Info Disclosure	MEDIUM	Partial	Env-var injection + key rotation + scoped SAS rules

ID	Threat	Category	Severity	Mitigated?	Control
T-DOS-AZ1	Azure transport saturation	DoS	MEDIUM	Partial	Azure throttling + bounded inbound batches + Service Bus DeliveryCount/DLQ
T-ELEV-AZ1	Blob container public access (incl. Service Bus blob sidecar)	Privilege	HIGH	By design	Private access default + Azure policy

Azure Transport Security Considerations

When using any Azure transport provider (Azure Queue Storage, Azure Blob Storage, or Azure Service Bus), the following security measures should be implemented:

Common to all Azure transports

- **Private Endpoints:** Configure Azure Private Endpoints for storage accounts and Service Bus namespaces so queue, blob, and AMQP traffic stays within the Azure backbone network.
- **Network Rules:** Use Azure Storage firewall rules and Service Bus IP/VNet filters to restrict access to specific IP ranges or virtual networks.
- **Key Rotation:** Rotate storage account access keys and Service Bus SAS keys on a regular schedule. Update the plugin configuration after rotation.
- **Monitoring:** Enable Azure Monitor diagnostic settings, Storage Analytics, and Service Bus diagnostic logs to track access patterns and detect anomalies.
- **Least Privilege:** For Storage, consider scoped Shared Access Signatures (SAS) over full account-level connection strings. For Service Bus, prefer scoped SAS rules over the namespace-wide `RootManageSharedAccessKey`.

Azure Queue Storage and Azure Blob Storage

- **Auto-provisioned containers/queues:** The plugin auto-creates the configured queue and container with private access if they do not exist. Enforce "no public access" at the storage

account level via Azure policy.

- **Single-container footprint (Azure Blob):** The Azure Blob provider stores both batch WAL blobs and deferred file blobs in the same container. Audit access policies and lifecycle rules on that one container.

Azure Service Bus

- **Pre-created queues with broker-side tunables:** The plugin does not auto-create Service Bus queues. Operators provision the queue with a chosen `LockDuration` and `MaxDeliveryCount` in the Azure portal, ARM, or Terraform. These broker-side properties cannot be set from a client; placing them under the same change-control as other Azure infrastructure keeps message-handling SLAs and DLQ policy auditable.
- **SAS connection string handling:** Service Bus authentication is via a SAS connection string of the form `Endpoint=sb://<namespace>.servicebus.windows.net/;SharedAccessKeyName=<rule>;SharedAccessKey=<key>`. The provider strips `SharedAccessKey=`, `SharedAccessSignature=`, and `sig=` query parameters out of any error messages it logs (`sanitizeServiceBusError()`) so SAS material does not leak into operator-visible logs.
- **Dead-letter sub-queue (DLQ):** Service Bus auto-moves messages whose `DeliveryCount` exceeds `MaxDeliveryCount` to the queue's dead-letter sub-queue. The plugin does not inspect or process DLQ messages; operators inspect the DLQ via the Azure portal, `az servicebus queue message`, or Azure monitoring. The plugin emits a WARN with error code `26004` when `DeliveryCount > 1` so poison-message buildup is visible before DLQ moves happen.
- **Independent blob sidecar credentials:** When `file_transfer_enabled` is true, file attachments transit an Azure Blob Storage sidecar that takes its own storage account credentials, separate from the Service Bus namespace. The two stores can live in different Azure subscriptions and have independent access policies. Audit and rotate the two credentials independently.